

Business Modeling with UML: A Business Process Centred Architecture.

Introduction

UML notation and use case centred architecture for developing software systems are considered to be the industry standard for OO system development. When it comes to modeling the business though, the situation is still far from being well established. Two mainstream architectures applied in business modeling are business use case and business process centred architecture. The first one extends the software system use case concept to model the business system while the second one is focused on business processes.

Modeling the business with use cases is comparatively new approach that has yet to find its way into business community while modeling the business by describing its processes is widely accepted and well understood by business analysts.

The purpose of this paper is to define an UML-based, process centred Business Modeling Architecture (BMA) that allows the business analyst to model the business 'as usual' (i.e. its processes first) with applying comparatively simple set of UML concepts. The resulting artifacts can be then smoothly transferred into the system analysis phase with its classic, use case centred architecture model.

As illustrated by a number of examples following our BMA description, the presented method of modeling the business takes the traditional approach based on business processes and uses comparatively new UML-based notation to define its deliveries. It does so in a coherent, simple yet powerful enough way for a purpose of modeling complex business system and its supporting software. In many aspects the method can be considered an example of Agile Modelling (AM) approach [2] as it combines several UML based modelling techniques in an agile manner around well-defined business modelling process with the sole goal to streamline the development process as it moves from business to system modelling phase. Instead of taking sides in a new "methodology war" of agile versus traditional approaches the article aims at amalgamating the best of the agile and traditional approaches [7]. In author's opinion, this kind of method is exactly what is needed to address the need to define an overarching business and system modelling method that stands a chance to be accepted by both sides of the business/IT equation.

Defining the Problem

UML is a standard notation for visual modeling of software systems and is widely used in OO systems development across the world. One of the reasons of the widespread success of UML is that it is methodology independent and can be used to express the results of projects of different size and purpose. In particular, UML can be used for business modeling and modeling of other non-software systems. In fact, business modeling with UML can be considered an extension of UML-based modeling discipline, related to system modeling using the same notation. As emphasized by many authors and practitioners [3, 4, 8], using the same visual language throughout the entire life cycle of the project development provides a great advantage for all project stakeholders making the development process more efficient. By having the business analyst and system developers using the same modeling concepts, the risk of costly errors related to different understanding of methodology concepts is significantly mitigated.

However, as pointed out by several methodologists, "The use of similar modeling constructs for several purposes can lead to confusion" [11]. Indeed, using UML as a notation for business modeling raises some fundamental methodology problems that we

will examine in the article in more detail. At the root of these problems is the fact that: “UML was defined to model the architecture of software systems” [5] rather than business systems. In a way, it was ‘reverse engineered’ from object-oriented programming concepts to model software through a series of visual diagrams grouped into several kinds of views describing software system behaviour and structure [15]. In most UML-based system architectures, the pivotal kind of view is the use case view of the system, which provides an external view of the system behaviour in response to user actions. The entire live cycle of the project is use case-driven with test cases traced back to their corresponding use cases for relevance [17]. Formally: “The use case construct is used to define the behavior of a system or other semantic entity without revealing the entity’s internal structure. Each use case specifies a sequence of actions, including variants, that the entity can perform, interacting with actors of the entity” [19]. UML notation of Actor and associated Use Case is shown on Fig. 1. To differentiate symbols used for system and business modeling, UML extensions called stereotypes are usually defined by a modeler: Actor, stereotyped as <<business actor>> may become a Business Actor and Use Case, stereotyped as <<business use case>> may become a Business Use Case as shown on Fig. 2 (both stereotypes and their icons are methodology specific [4] i.e. not defined by UML itself).



Figure 1. Actor and its associated Use Case in UML notation.



Figure 2. Stereotypes of Actor and its associated Business Use Case in UML-base notation.

In almost all methodologies a use case is expected to yield some observable result of value to an actor in order to complete a business process. However, for the reasons described above, UML does not define a business process precisely saying only that: “A use case model is a model that describes the business processes of a business and their interactions with external parties such as customers and partners” [19], and indicating that activity diagrams can be used to model business processes:

“Activity graphing can be applied to organizational modeling for business process engineering and workflow modeling.” [19]

As with use cases, UML does not define any business stereotypes of Activity Diagram elements, which primary mean is to model system behavior:

“An activity graph is a special case of a state machine that defines a computational process in terms of the control-flow and object-flow among its constituent actions.” [19].

As a notation, UML does it specify any methodology interdependencies between use cases (system or business like), Activity Flows and Business Processes. UML does not even require any particular business extensions of its standard notation should be used to describe business systems. UML specification 1.4 [19] includes a “Standard Profile” for business modeling, but it is not recognized by the business as an industry standard. The Profile uses Package and Entity extensions to denote several business concepts (business process is not among them, though) and can be considered just one of many ways of extending UML for the purpose of business modeling. For example, Business Actor and Business Use Case extensions from Fig. 2 are not defined in the Profile. From the business perspective, the Profile “describes business modeling extensions in terms of OO concepts with which most business users are unfamiliar.” [11].

With such an imprecise UML description of a Business Process and its relation to Use Case and Activity concepts business analysts of today embarking on UML-based business modeling project can find the relationship of the those concepts equivocal. Add to that the problem of transforming of business analysis deliveries into system requirements and the risk of getting it all wrong is substantial, even with disciplined approach. Trying to resolve the problem by referring to leading business modeling methodologies may only augment the confusion as there are several totally different approaches to the problem and none of them can be considered an industry standard at the moment. The stalemate that has resulted from this situation and lack of satisfactory solutions both within UML itself and on the side of methodologists that are trying to tackle the problem has lead some of them to claim that “UML does not allow you to model your business. Use cases are software specific and too unwieldy for serious business process analysis.” [10].

The aim of the paper is to address this problem by defining an UML-based Business Modeling Architecture that describes the business processes in a precise way but separates Business Modeling Discipline from Use Case Analysis, which comes to the picture only when system requirements can be effectively derived from business analysis deliveries. As illustrated by a number of examples following our BMA definition, the presented method of business process analysis is simple enough to be applied in a business modeling project yet powerful enough to model complex business systems of any size. Moreover, it provides a sufficient input to move the project to the system analysis phase. In author’s opinion it is exactly what is needed to address the problems that have rendered business modeling with UML into its current situation.

Business Process and its Environment's Meta-model

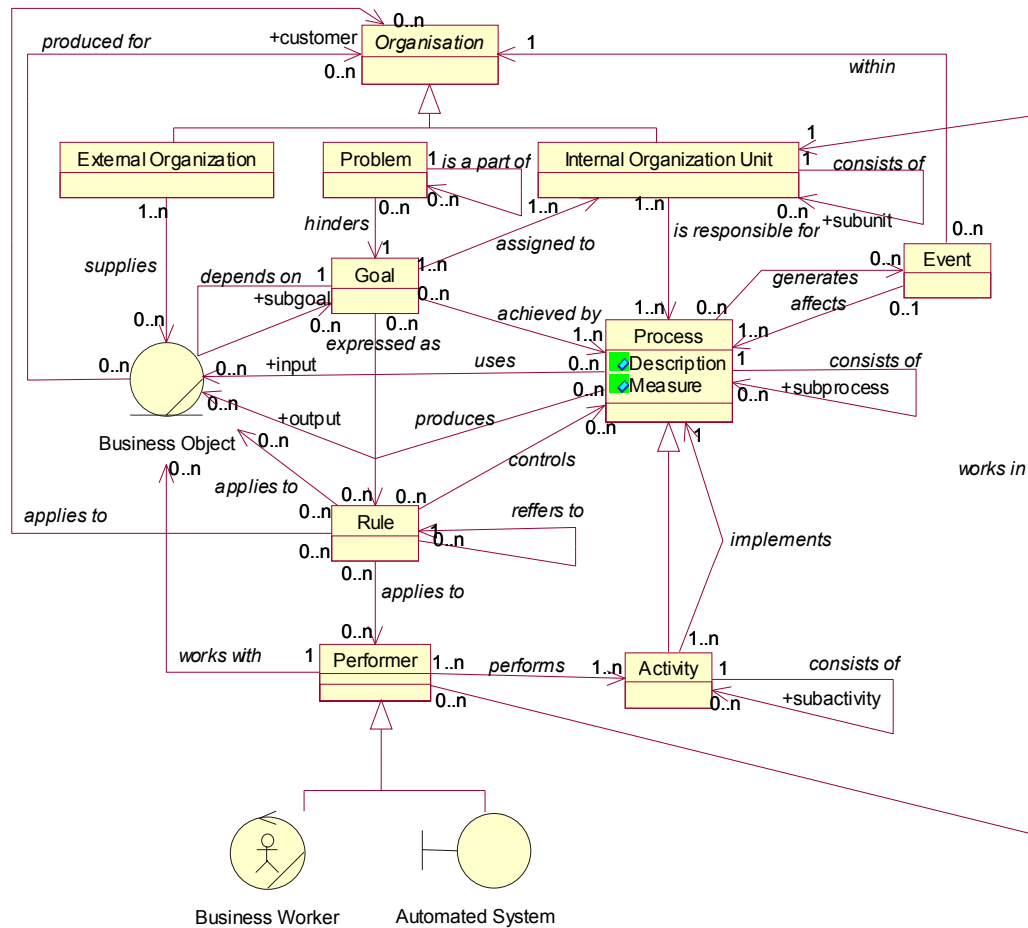


Figure 3. Business Process and its Environment's Meta-model

A number of Business Process definitions have been proposed by different authors, which emphasized similar aspects of the term. A summary of similarities of these definitions can be found in [6]. Based on that, we define Business Process as an abstraction of active part of the business that describes how the work is performed in business environment.

One of the best ways to describe a general concept is to model it and its relationships to other concepts in a form of meta-model used as a pattern to create specific business models. Fig. 3 is a meta-model centred on Business Process concept. This meta-model is a UML class diagram in which each concept is depicted as a meta-class, with stereotype icons used in some cases. The relationships of the modeling concepts are either an association or specialization. The meta-model shows only the most important attributes of Business Process.

The meta-model in Fig. 3 shows following characteristics of the Business Process:

Description: usually textual, generic description of how the process executes, which is easily understandable for people familiar with the business. The description should specify, how the input objects of the process are transformed into output objects or consumed by a process. The transformation of the input objects may be physical, logical, transactional, or informational.

Measure: established standards that reflect customers' expectation for product and service quality, quantity, timeliness, or cost.

Goal: each Business Process needs specific goal that reflects business expectation of its outcome and performance possibly expressed by defined Measure. Process Goals should be aligned with Organisational Goals up to the highest level of Organisation

Strategy: all the processes collectively attempt to achieve the strategic goals of the business. Each Goal can have a number of subgoals it depends on and some Problems that hinder its achievement.

Input Business Objects: Business Process may use a set of input objects (such as materials and information) and transform or consume them in the process to produce transformed or new process output.

Output Business Objects: output objects are primary outcome the process produces and represent the accomplishment of Process Goal. Output objects from one Process can be an input for another. They represent value to some kind of internal or external customer.

Business Rules: Business Rules are statements that can control the execution of the Business Process, affect the structure of the Organisation and its Business Objects and constrain the behaviour of Business Actors. Some Business Rules express Business Goals. Business Rules represent business knowledge and can be categorised as functional, structural and behavioural and can refer to each other. Business Rules are expressed through various elements of business models such as inheritance, association types and multiplicity, persistence of Business Objects, statechart diagram structure and many others. A formal modeling of Business Rules (e.g. with Object Constrain Language recommended in UML) is out of scope of this paper.

Business Process environment: business processes are not executed in a vacuum – each Business Process has at least one Internal Organisational Unit responsible for its management and control. Many Business Processes are triggered/affected by some kind of Business Event outside or within of the organisation or generate such Events themselves. Business Objects might be produced for or supplied by external organisations (such as clients and suppliers, respectively) but processes of those external organisations are outside our modeling scope, even though some of them can be external Business Events (such as 'Place Order') that trigger a particular Business Process. Obviously, triggering Business Events might also happen within the organisation (e.g. 'Request Report').

Subprocesses: Business Process can consist of a sequence of other processes characterised in a same way. As pointed out by E. McSheffrey [11]: "The concept of process is intrinsically hierarchical. Processes may be defined at any level from enterprise-wide processes to processes performed by a single person. Low-level processes may be grouped together to achieve a common business goal. To avoid modeling to an inappropriate degree of detail we must identify the lowest level of process, which is needed at the Conceptual view. This is usually taken to be the

Elementary Business Process (EBP).” In the paper, we define EBP as a process performed in one location at one time, which adds significant value to the business and leaves it in a consistent state. EBP can’t be subdivided into smaller subprocesses i.e. it has to be completed or not done at all.

Note that, even though it’s often the case, we do not require EBP to be performed by ‘one person’ – this is because, as shown in the meta-model, processes are assigned to Organisational Units rather than Performers, therefore we leave this type of requirement to the activity level of analysis.

Activities: the important characteristic of the business processes defined above is that we do not require a Business Performer to be assigned to a particular Business Process. Instead, each Business Process runs within one or more Internal Organization Units responsible for its execution.

The most detailed level of business modeling is the activity level, where each step of a Business Process is assigned to a Performer working as a part of the business with a set of Business Objects to provide the expected outcome accordingly to the Goal of the Activity. Work of the particular Activity can be done by a Business Worker (a person) or an Automated System that are both subclasses of Performer.

An Activity is a planned or organized thing to do that is performed by one or more Performers. Activities implement a Process. As the execution steps of Business Processes, activities are characterized in the same way, only at a more granular level. Similarly, an Atomic Business Activity (ABA) is an activity performed by one Performer in one location at one time, which moves forward the business process, and which can’t be subdivided into smaller subactivities of the same characteristic. ABA is often called a task or a process step.

Note that Activity does not have to leave the business in a consistent state as it can be only a step in an EPB e.g. ‘Entering the card into ATM machine’ is an example of such ABA which might be a part of ‘ATM withdrawal’ EBP.

Types of Business Processes

Looking at the specifications of different business processes, it’s possible to identify at least three types of them:

First, there are the commercially important processes that result in a product or service received by the organisation’s external customer. These are usually called primary or core processes. Processes such as Order Fulfilment or Warranty Administration are good examples of these.

Second, there are processes whose outcome is not visible to the external customer, but that are essential to the effective functioning of the business. These processes are called support or enabling processes. Examples include: Budgeting, Recruitment, Security, and System Administration.

The third category of processes – management processes – describe the work of the managers to support the business processes. These processes affect how the other processes are managed and the business’ relationships to its owners. Examples would be: Strategic and Tactical Planning, Goal Setting, Operations’ Review.

Business Modeling Architecture

Architectural models define the business structure and are the key to understanding the business and its functionality. As stated in [6]: “A good architecture allows the modeller

to abstract the business into different aspects or views and to concentrate on only one aspect at a time.”

Business Modeling Architecture is focused on business models that can be created in a form of UML diagrams and does not include ‘soft’ views of the business such as ‘Cultural’ or ‘Capacity for change’ views – these are covered by a broader term of Business Architecture.

The Business Modeling Architecture views and their interdependencies described in the paper are directly aligned with the meta-model presented in Fig. 3 and are shown in Fig. 4. Each view is realised by a number of models, sometimes complemented by a textual document. The diagrams can be linked with each other to allow for easier navigation through the entire model.

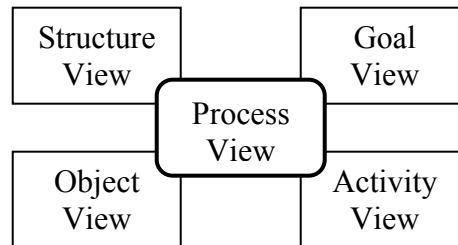


Figure 4. Business Modeling Architecture

The business modeling architecture described in the paper is consistent with Model Driven Architecture (MDA) specification [12] adopted by OMG. In particular, the following guidelines are followed:

“The MDA leverages UML models as abstractions, different viewpoints, and different levels of abstraction.”

“There could be more and less abstract models within each of the business, PIM¹, or PSM² levels.”

“MDA permits zooming in to result in multiple alternative models (and vice-versa) by separating the refinement model that relates the detailed and simplified views.”

More detailed description of OMG’s MDA is beyond the scope of this paper and can be found at OMG’s Web site. An overview of MDA as an architecture which provides a foundation of business modeling connected to technological realm, you can refer to [20].

Process View

Business Process View is at the centre of Business Modeling Architecture. The view demonstrates the interaction between Business Processes of different levels executed within Organisational Units in order to achieve Business Goals. The Process View illustrates the resources interacting with the processes and the value outcome produced as a result. It also shows the Business Events affecting/triggering some of the Business Processes. Additionally, the hierarchy of Business Processes broken down to the contributing Subprocesses of lower level can be shown on a separate diagram.

The Business Process View is modelled with UML Activity Diagram showing Processes as stereotyped Activities and Organisation Units as swim-lanes.

¹ Platform Independent Model

² Platform Specific Model

A high-level process map of an example organisation is shown in Fig. 5. Processes taking place outside the organisation are stereotyped as <<external>>. Processes that trigger business response on the organisation side are stereotyped as <<event>>. ‘Place Order’ is an example of business event that triggers ‘Place Customer Order’ process in the Marketing and Sales organisation unit by sending a ‘Customer Order’ business object to it. Marketing and Sales Processes of Level 0 have been already broken down into Level 1 Processes as shown in the process hierarchy in Fig. 7. ‘Place Customer Order’ Process can be broken down into next level workflow as shown in Fig. 6. Some of the Subprocesses shown on this diagram might be EBP (like ‘Receive Customer Order’); others (like ‘Correct Customer Order’) can be modelled at yet more granular level. Problems, Goals and Rules can be attached to the corresponding processes in the workflow diagrams.

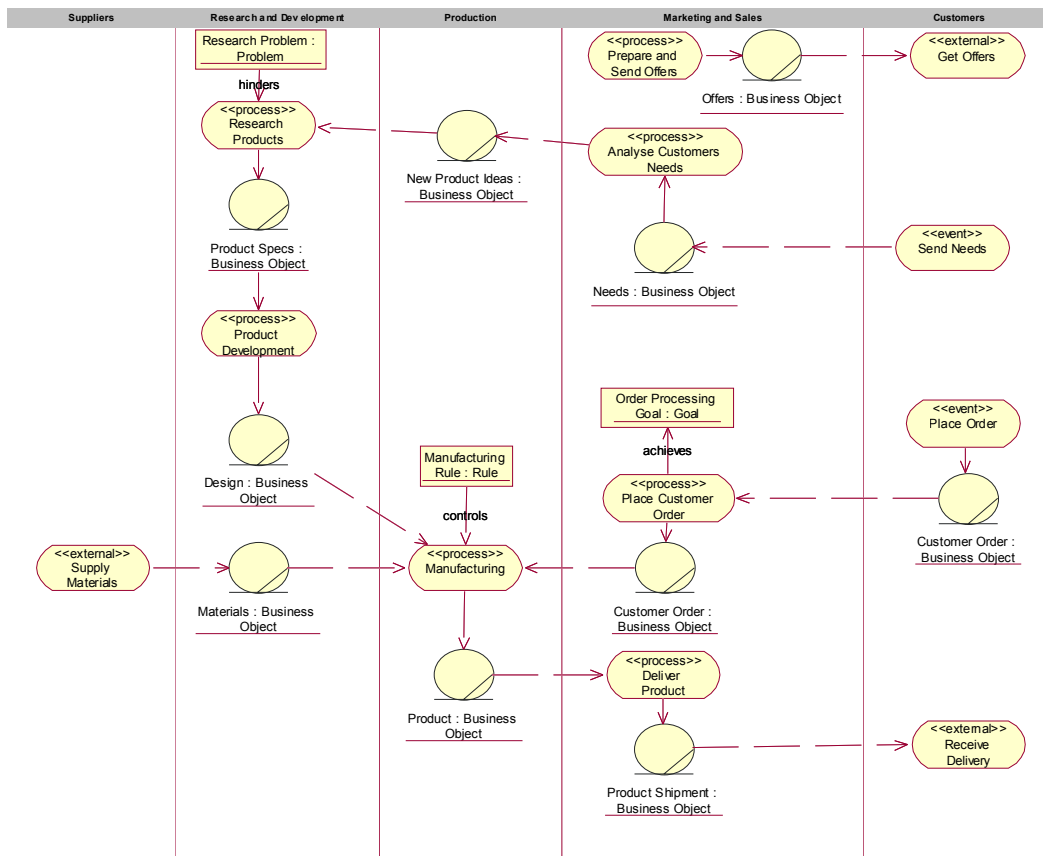


Figure 5. Example of the Organisation Process Map

Note that so-called Context Diagram can be obtained by contracting Level 0 Process Map to show only Business Events and Processes triggered by them in the organisation.

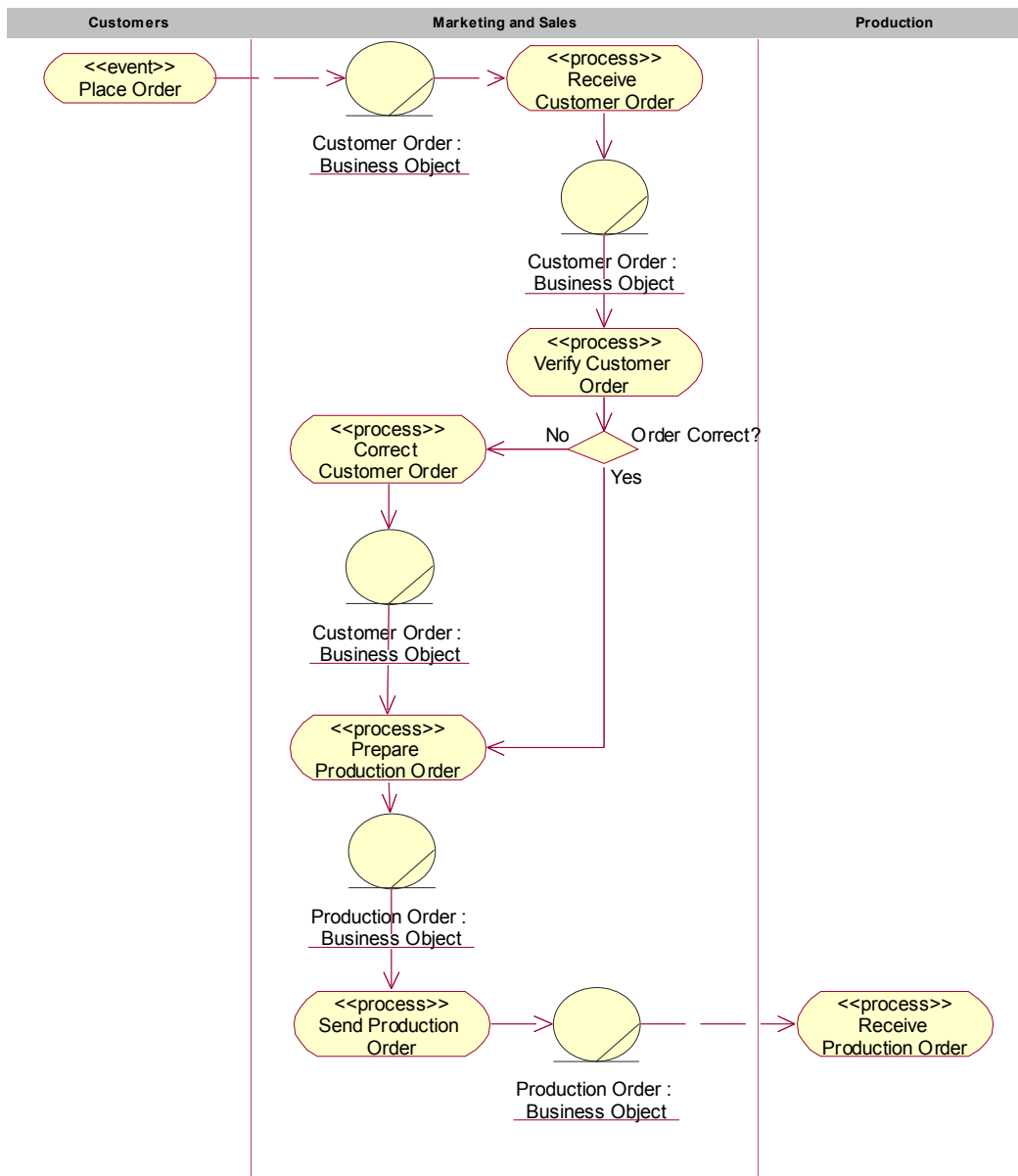


Figure 6. 'Place Customer Order' Process Workflow

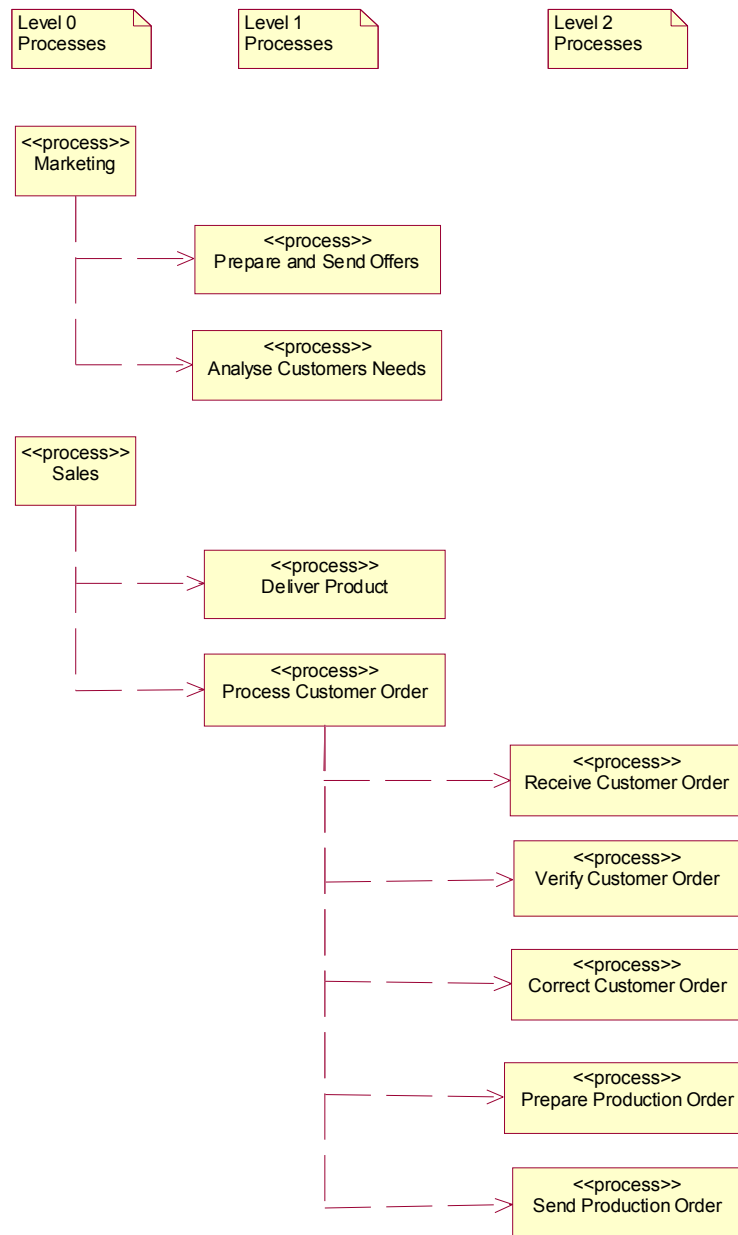


Figure 7. Process Hierarchy

Goal View

A business organisation is a value-driven system with high-level mission statement expressed by organisation Goals achieved by core Processes with measurable Goals of their own. Activities of each Process might have specific objectives (Goals) implementing Process Goals. As a result we can create a goal structure covering three levels of the business: organisational, process and activity.

At the organisation level, Goals are part of the business strategy, which in turn along with the company mission is an important factor of a broader business vision view containing

several other factors such as strengths/weaknesses, opportunities/threats, core competencies etc. [5].

At a high level, all views of BMA contribute to business vision view. For example, organization Structure View provides a model for structuring the business into manageable subdivisions.

Process Goals build on direction established by organisational/strategic Goals. The Goals and Measures of core Processes should be derived from organisational Goals and customer needs. The Goals of support Processes should be driven by the requirements of core Processes to achieve their Goals. Management Processes allow for smooth operation of other processes aimed at achieving theirs and organisational Goals.

Just as we need to establish Process Goals to achieve organisational Goals, we need to establish job objectives for people and automated systems that perform the work activities. Measuring whether we actually meet these objectives is the only way we can effectively manage the business and its employees and make operational and strategic decisions.

A goal model describes the Goals of the business at different levels and the Problems that hinder achieving the Goals. It may also indicate ways to improve the business or resolve conflict between the Goals. Shared with the Business Workers it can also be used as a motivation tool in realization of the business vision.

The Goals and Problems are modelled in a UML goal/problem diagram shown in Fig. 8 as a class diagram. Specific Goals and Problems can be created as objects of the classes shown in there.

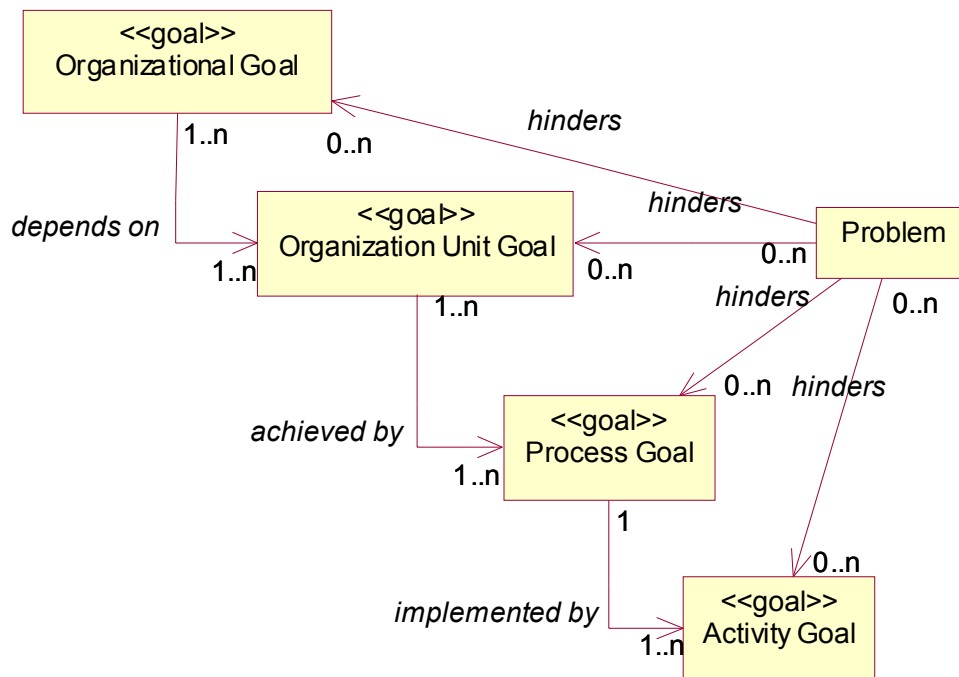


Figure 8. Generic Goal/Problem Class Diagram

Other technic of building the goal model is to use classes defining quantitative/qualitative Goal types and objects of those classes to show the actual Goal structure with Problem notes attached to them [5].

From the architectural perspective we are taking here it does not matter which modeling technic is used as long as it corresponds with Business Modeling Architecture defined above and is aligned with its meta-model shown in Fig. 3. The same applies to other models presented in the paper. This flexibility in modeling technics that can be used emphasizes the fact that the fundamental concepts defining Architectural Framework of Business Modeling are its view structure (Fig. 4) and conceptual meta-model (Fig. 3).

Structure View

The Organisation Structure View is supplemental to the Process View and shows the organisation of the company into organisational Units such as divisions, departments, sections, and so on. These same units are used to divide process diagrams into organisational swim-lanes, so the two views contribute to each other and must be consistent. Structural View not only shows the static structure of the organisation, but also contributes to understanding how the work gets done and whether it makes sense to organise the company in this way in a context of its business Processes. At activity level Structure View can show Business Performers (people and machines) operating within a particular Organisational Unit.

As before, class and object diagrams can be used in UML to model structural view of the organisation with classes defining organisation unit types of subsequent layers and objects describing the actual structure.

The structure of our example company complementing the Process View from Fig. 5 and 6 is shown in Fig. 9 with packages stereotyped as Organisation Units used to show its simple structure. The business Performers (Workers and Automated Systems) can be then grouped into Organisation Units (Packages) they are assigned to work in.

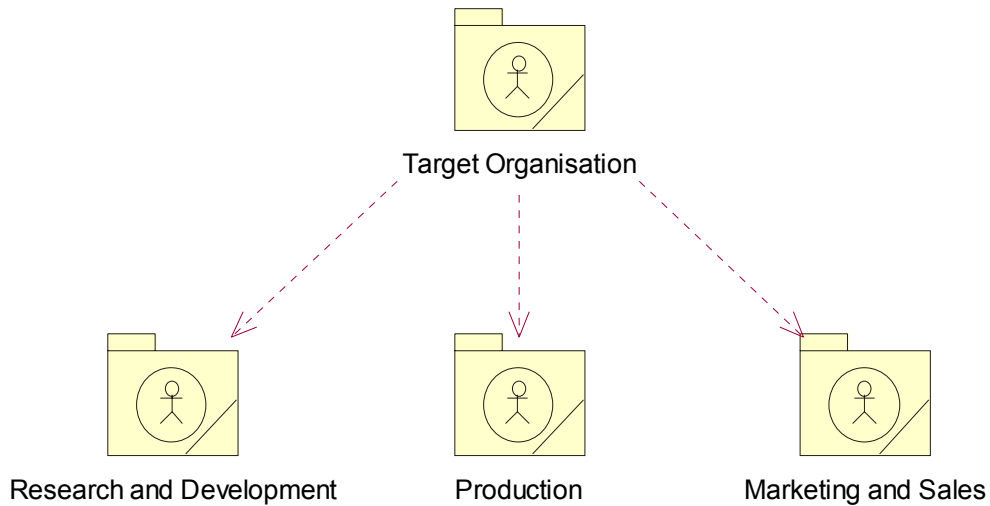


Figure 9. Structure View of Example Company

Activity View

While organisational outputs are produced through processes these in turn are performed by individuals (Business Workers) or machines (Automated Systems) doing various jobs. As emphasized in a classic book of G.A. Rummler and A.P. Brache [18], which actually launched the Process Improvement Revolution:

“ If jobs are not designed to support process steps, and if job environments are not structured to enable people to make their maximum contributions to process effectiveness and efficiency, then Organization and process Goals will not be met”

Activity Diagram shows the flow of work between Activities and the Jobs (Performers) to which the Activities belong. This aids in identifying the areas where performance should be improved. Activity Flow Diagrams can be created in the context of a Process

and its responsibility to an Organizational Unit in a Process Diagram. The activity flow diagram would be a further refinement of the Process.

To illustrate our concept, let us consider 'Place Customer Order', which is a core Process modeled in Fig. 6. Let's say that the process Goal here is to 'Minimize processing time' and it contributes to an organization Goal of 'maximizing customer satisfaction'. It's pretty clear that if we don't design jobs with their responsibilities structured around the Process in its actual business environment it's rather unlikely that those Goals will be met up to the company potential (and minimizing its operating costs) not mentioning the worst case scenarios; and all that is needed in spite of how optimal our final process design might be.

Let us then assume that our customers will be placing orders by sending paper forms by regular mail, calling our toll-free number or shopping at our top-of-the-line Web site. To handle all this traffic we will place a dedicated mailroom operator to bring the mail from our mailbox at the post office, open and sort it for easier handling by a Customer Order Specialist (COS). COS's job will be to verify and possibly correct the orders and send them to the Production Order Specialist to prepare and send a detailed Production Order which may require additional information/processing such as ordering components, scheduling etc. The orders taken by our phone line representatives will be already verified, so we can send them directly to the Production Order Specialist. The same scenario (at least for simpler order types) is envisaged with our user-friendly Web site. The resulting diagram is shown in Fig. 10. Actually, this diagram is a compound of three different activity diagrams corresponding to three means of placing orders by the Customers. For simplicity we use just one Customer Order: Business Object in all business flows which, after detailed analysis, might not be the case. As mentioned before, some of the more complex Activities, like Verify or Correct Customer Order, probably should be analyzed at a more granular level. Each Performer and/or Activity might have Goals assigned to it that correspond to the overall process Goal specified above. For example, the goal of Customer Order Specialist job might be to process a certain number of orders per day (i.e. with a premium for topping it).

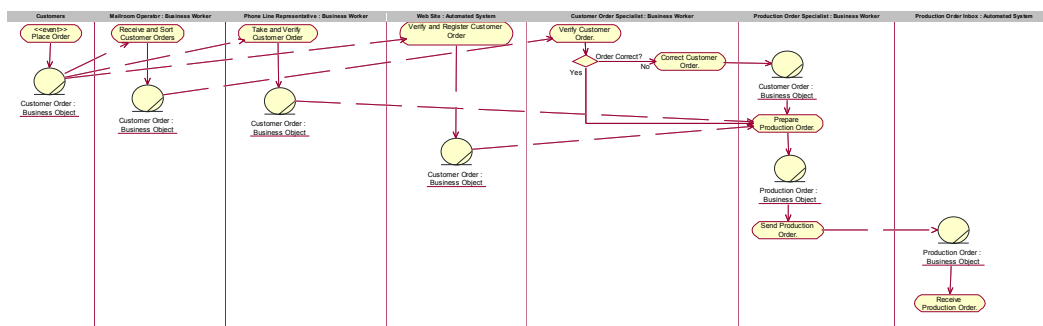


Figure 10. Place Customer Order Activity Diagram

From the example described above we can see that the activity level is where the greatest potential of work improvement is (assuming a well thought-through design of the underlying Process). It can be realized by organizing work in an efficient and motivating manner. For example, gathering 'traffic' statistics we can put enough resources (both people and hardware) to handle all channels of order processing. This is yet another example supporting the well-known statement that 'If you can't measure it, you can't

control it; if you can't control it, you can't manage it'. Management of business Processes is a related subject that is out of scope of this article – our intention here is to emphasize the importance of the Measures assigned to the Processes and Activities in our meta-model.

In summary the Activity level:

- describes in detail the Activities each Job/Performer is responsible for and the Goals to be achieved
- provides a “micro” picture of Performers and their immediate environment.

Technically, activity diagram looks much the same as process diagram only at the more granular level – swim-lanes correspond to Performers (jobs) rather than Organisational Units, and Activities describe what the Performers have to do to get the job done.

Object View

The Object View shows the structure of Business Objects and the way the Performers interact with those objects to produce process outcomes.

Business Objects can be described textually or by specifying Attributes (or associated Business Objects) and modelled in UML class diagram. Business Performers can be placed on the same diagram to show collaboration of participating Performers and Business Objects (Entities).

A business class diagram showing relationships among Business Entities and Performers participating in the 'Place Customer Order' Process is shown in Fig. 11.

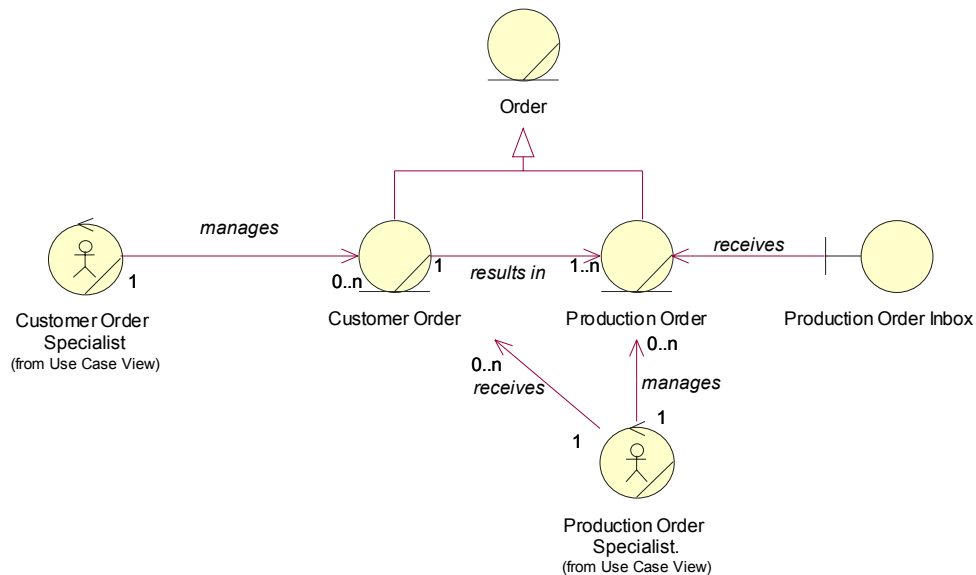


Figure 11. 'Place Customer Order' Business Class Diagram

Customer Order and Production Order are defined as business Entities, which are both subclasses of abstract business entity class: Order. One Customer Order results in at least one Production Order. Customer Orders are managed by a Customer Order Specialist. Production Order Specialist receives Customer Orders and manages Production Orders. Production Order Inbox is a part of the automated system that receives Production Orders. Note that diagram from Fig. 11 is consistent with the activity diagram from Fig. 10 but shows the Business Entities and Performers in different context.

A detailed behaviour of business Entities can be shown on a state-chart and sequence/collaboration diagrams but creation of those is often delayed to the system analysis stage after decisions about process automation have been made and system classes representing business entities have been defined. An example of Customer Order state-chart diagram is given in Fig. 12. As we can see from this model, Customer Order can't be cancelled after it has been sent to production (expression of Business Rule).

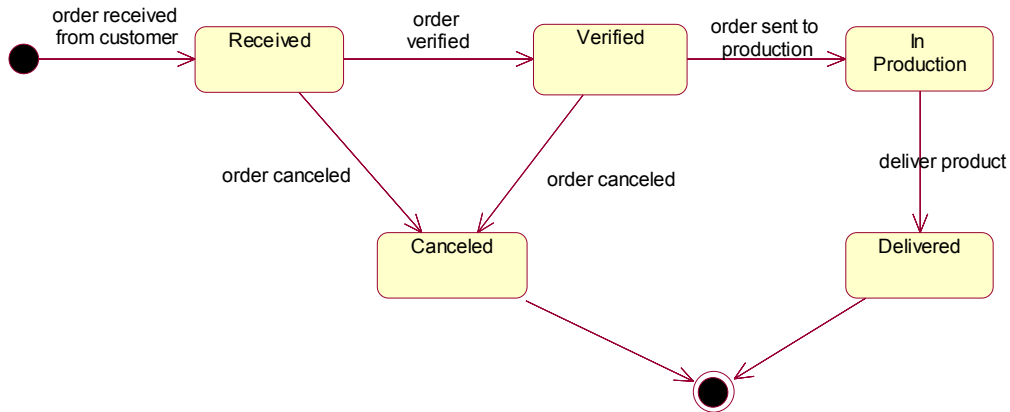


Figure 12. Customer Order State-chart Diagram

To conclude let us say that while Object View is using purely business concepts it shows them in system context by using terms such as Entities, Attributes, Associations, Inheritance etc. Detail analysis of these model elements permits business knowledge to be presented in a precise manner that can be verified by domain experts on one side and provides a solid input into system modeling – on the other.

From Business to Software Modeling

Transferring the business model into the software model is considered to be most difficult part of the analysis phase of the project and there are several reasons for that. First, the two models serve different purposes and are usually built by different teams. Second, the views of the architecture of software systems are different from those used to describe business system and there is no direct mapping from business to system architecture views. Third, there is no precisely defined method of how to translate the business model into software model. However, as we will see from our description of the transition process, understanding the purpose and structure of the two architectures and the purpose of the transition process itself, and dividing the work into three well-defined activities can smooth the transition process considerably and minimize the risk of ending up with inaccurate software model.

A 'classic', UML-based software architecture model consists of '4+1' views [15] shown in Fig. 13.

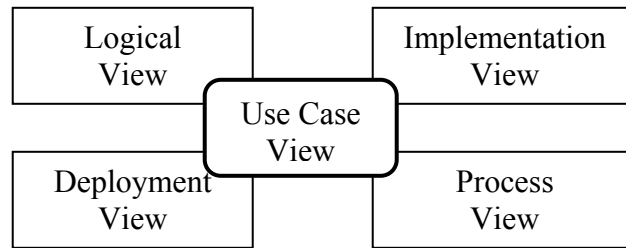


Figure 13. '4+1' Software Model Architecture

Even though several other architectural structures has been proposed e.g. Extreme Software Architecture described in [14], the central view of software architecture modeled in UML is usually the Use Case View which is central to the entire model and used to drive and verify the design of the IT system. Use case models are also the first ones to produce in the initial stage of the OO system development project and define the requirements of the IT system and its first cut architecture. Therefore, the logical way to proceed with transferring business models into system models would be to use business process models to create use case model of the IT system supporting these processes in a accurate and complete way. In order to do that, we need a 'bridging' type of model that will show us in detail how IT system is to support the already modeled Business Processes. We define this type of model as IT Supported Activity Diagram example of which is shown in Fig. 14.

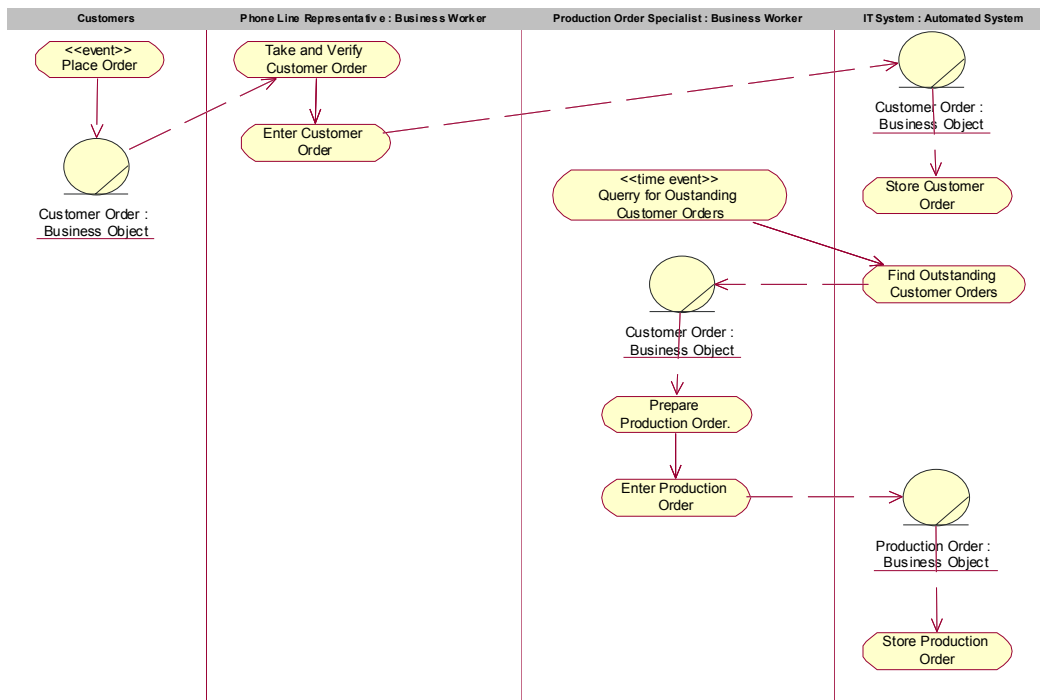


Figure 14. IT-Supported Activity Diagram

The diagram shows activities of the same 'Place Customer Order' process (order taken by phone) only with IT System swim-lane added to the flow shown in Fig. 10. As a result of that addition the flow of work has been automated: some of the manual activities has been eliminated/replaced by system related activities and system activities has been

added in its swim-lane. This model is the integration point between Business Process and System Use Cases to support it.

Grouping together system related Activities we can create the supporting system use case model shown in Fig. 15.

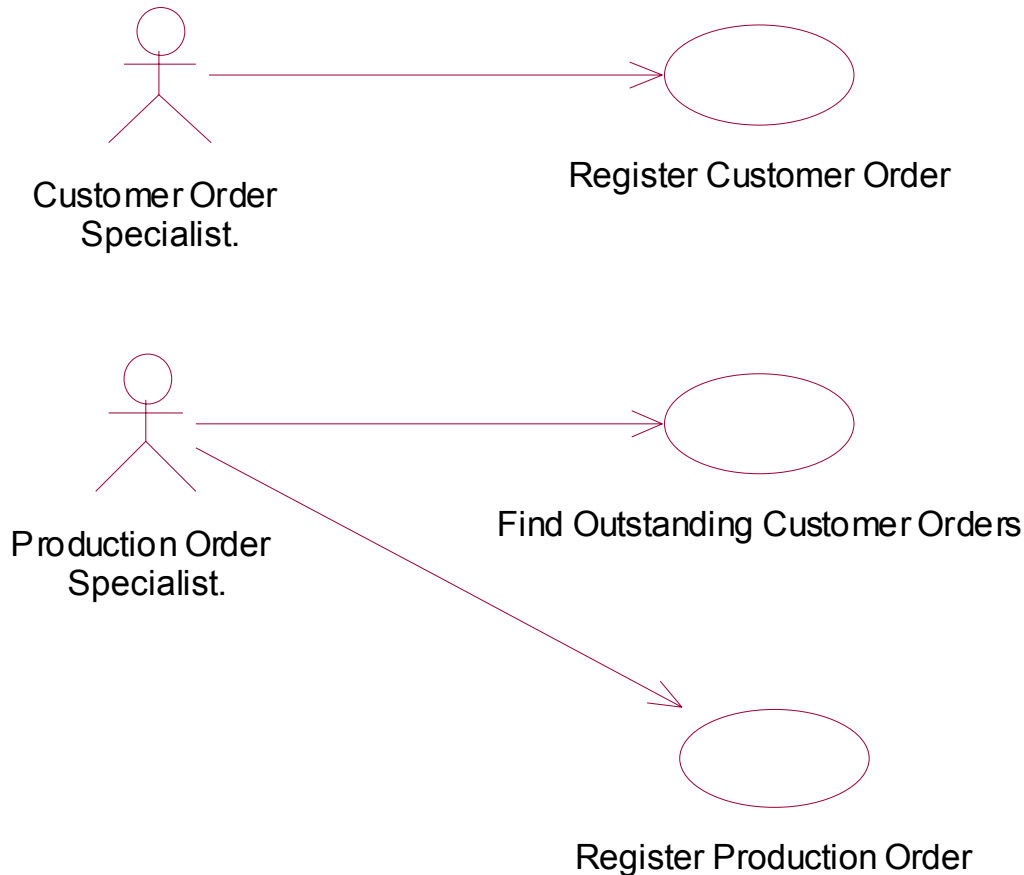


Figure 15. Use Case Model to Support 'Place Customer Order' Process

The model shows a group of Use Cases required to support the Process and the System Actors interacting with the IT system. Creating a complete and accurate use case model to support Business Processes is a second part of transferring the business models into system models. The description of 'Register Customer Order' use case (the success scenario if it) may look like this:

Example

1. Customer Order Specialist (COS) enters the Customer Order (CO) data into the system.
2. COS requests the system to register CO.
3. The system validates the data entered <E1> and verifies whether CO can be registered <E2>.
4. The system saves CO and confirms this fact to COS.

Exceptions:

<E1>: Data entered are incorrect. System displays error message.

<E2>: Business rules do not allow the order to be registered (e.g. 'out of stock'). Error message specifying the reason is displayed by the system.

With use case model in place we have fully transferred the analysis into system specification phase through use case modeling. To illustrate the power of this technique, we show in Fig. 16 the sequence diagram describing interaction of the System Actor and its three types of classes (boundary, control and entity class) in the realization of this use case.

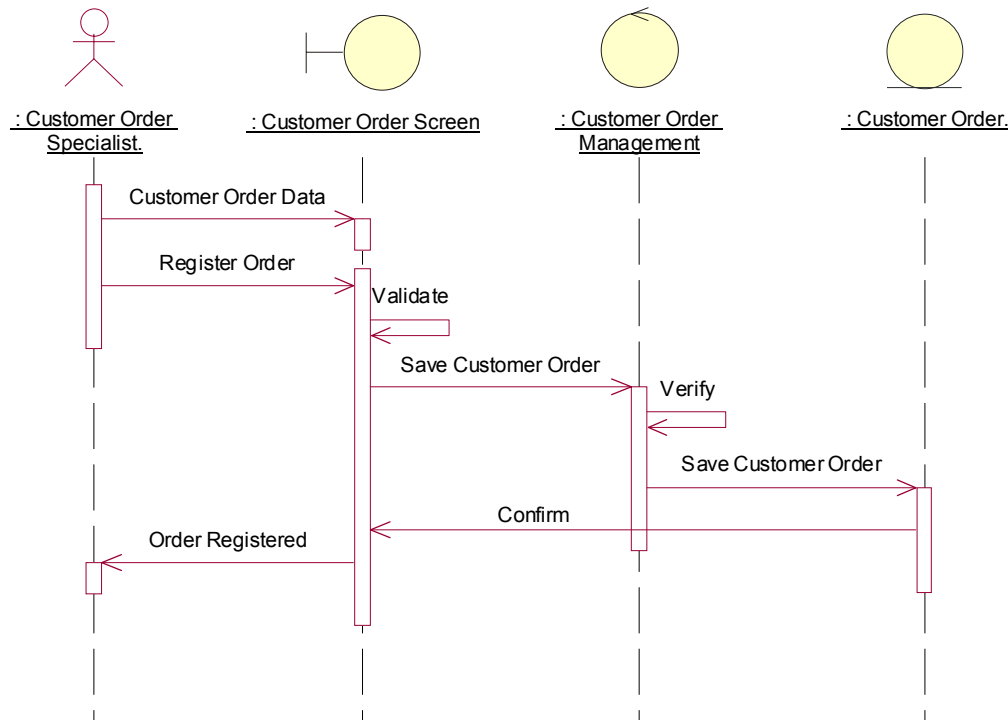


Figure 16. Sequence Diagram: 'Register Customer Order' Use Case Realisation.

At this point we have not only moved to system analysis phase – we are actually doing initial system design by using three-tier system architecture and applying use case analysis technique. The interface tier is represented by boundary class 'Customer Order Screen' and handles the system interaction with the user e.g. data validation. The business logic is controlled by Customer Order Management class that is responsible for more complex processing like order verification. Accessing the database is handled by the third layer of the system represented by Customer Order entity class. This type of diagram is called a use case realization and shows a design view of the use case. "The use case realization is the transitional element that specifies "how" the use case will be implemented." [17].

Now that we have described the activities and techniques of transferring our business models into system specifications we can generalize our observations of this process into the transfer model shown in Fig. 17.

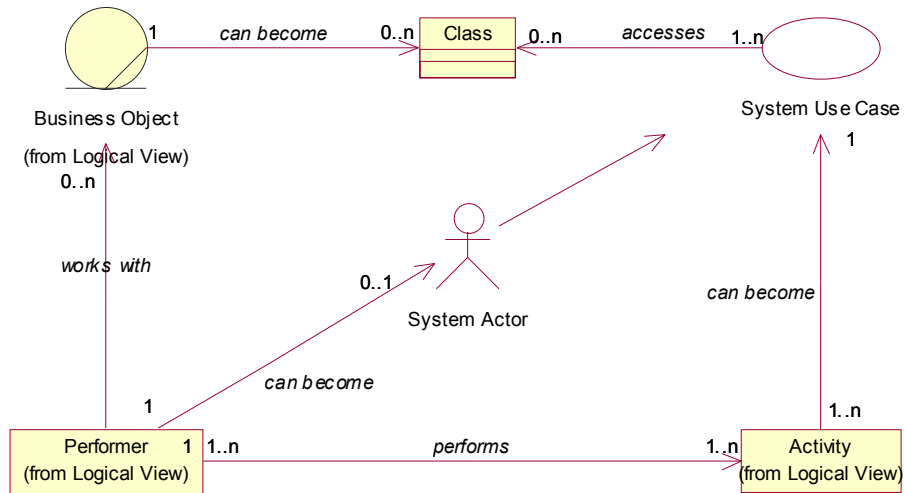


Figure 17. The Model of Business to System Transfer.

This model simply shows which elements of the business model can be transferred into corresponding elements of the system model. A group of system related Activities from the IT supported activity diagram, performed by the same Performer, could become a system Use Case with the Performer becoming a System Actor. Business Objects that the Performer works with can become entity Classes of the system, accessed (through boundary and control classes) in the flow of the Use Case.

Methodology Guidelines

Business Models are usually created for two primary purposes: improving the business processes by restructuring their flows or building the information system to support them. Combinations of the two are also common (e.g. e-business modeling) but require even more cooperation of the business and IT staff some of which should understand both business and system type of modeling. Depending on the main purpose of the business modeling project the methodology described in the paper can be customized to suit the project needs.

In the first case (business process improvement) the modeling is usually focused on selected core processes that require reengineering or do have the greatest potential for improvement. The classic approach is to build ‘as-is’ models of these processes and then, based on defined organization strategy and main goals create optimized, ‘to-be’ models of them. Depending on the scope of the project, its timeframe and resources, the modeling effort can go as deep as activity flows of the selected processes. In many cases, the greatest improvements can be made in the ‘white space’ [18] of the organizational flowcharts i.e. areas that cross organizational boundaries of the process flows. The optimization of the core processes can’t be underestimated – they create the main value output of the company and interact with its customers. With optimized core processes in place, we can then analyze whether the support processes are structured in a way that streamlines the flow of work of the core processes. With efficient processes in place we can then design how to control and manage them effectively with the management processes of the company, for with clearly defined ownership and measures can be put in place.

As IT system acts as an amplifier of the effects of the business process [21] it is critical to the success of the IT project to provide software solution to a process that makes sense from the business perspective. Otherwise, the main result of the IT project might be the acceleration of the collapse of the process itself with possible scapegoat being the IT system itself (“before its introduction everything was just fine”). However, when the explicit purpose of the business modeling is to find requirements for the supporting IT system it is important not to over-model by describing details of the business which are not relevant to the IT system [6]. In our case this can mean modeling the processes that the IT system is planned to support in order to understand them from the business perspective and make sure that IT automation will increase their efficiency. In order to achieve this, activity models are not always required and the process modeling usually stops at the level that is sufficient to determine which process, typically run by one Performer, can be automated by IT system. Based on that, use case models can be produced and further discussed with the business experts to confirm they are supporting the right business process in a way required.

For example, analyzing business process ‘Place Customer Order’ (Fig. 6) we may decide that the IT system will support two of its subprocesses: ‘Receive Customer Order’ and ‘Prepare Production Order’. By analyzing those two subprocesses further and optionally – creating IT supported process diagram, we can come up directly with the use case diagram shown in Fig. 15. We can then discuss these use cases with the business expert who knows how the work is actually done, or how it’s planned to be done. In fact, this simplified method is the same as before, only the activity diagrams from Fig. 10 and 14 are assumed to be a part of the business knowledge of the business domain expert rather than analysis deliveries. As one can rightfully guess, this method requires a higher level of skills (and trust) on both business and IT sides of the project.

A short-cut approach to business modeling is the so-called domain modeling where only business objects are modeled and transferred to entity class model, then - the use cases accessing them are used to verify the business processes they are supposed to support. This is even more radical approach as it actually alters the defined Business Architecture (Fig. 4) by making the Object View its central part. In our example it would mean creating the object diagram shown in Fig. 10 (with some statecharts like the one in Fig. 11) and processing straight to system analysis from there. This kind of approach can be used when business domain is well understood and there is a close cooperation of the business and IT sides of the project/organization with business domain experts being able productively discuss system-modeling concepts with system analysts and vice-versa. Depending on the project environment, various other modeling techniques can be applied for the purpose of defining accurate and complete system requirements with GUI prototypes being one of users favorites. This is where Agile Modeling can come handy [2], but only if a solid and well understood project methodology is put in place first as otherwise any AM attempts can only amplify any tendencies in the team members to model it ‘their own way’ which often results in a ‘Babylon Tower syndrome’ of model deliveries of not in pre-analysis paralysis (analysts talking about what types of models should be produces instead of actually working on them).

Business Process vs. Business Use Case Centered Architecture

One of the noticeable facts about the business modeling architecture and underlying methodology presented in the paper is that it does not make use of Business Use Case

concept in any way. Wouldn't it be beneficial to have the same modeling constructs applied in both business and system analysis as discussed at the beginning of the paper? In our opinion, most of the time – the opposite is true: use cases (stereotyped to business use cases as shown in Fig. 2) are not best suitable for a complex business modeling of any purpose (process reengineering or IT automation).

To illustrate our point of view let us consider two main types of business modeling methods that rely on business use case modeling (partially or in full).

The 'partial' type of method typically goes to process decomposition reaching EBP level and then defines one Business Use Case for each EBP performed with the support of the system [8, 10]. To detail it further, IT-Supported Activity Diagram (AD) is created which is a base to model the system use case scenarios with sequence diagrams.

In our example, this would mean decomposing the business processes to the level shown in Fig. 6, then defining a business use cases for each EBP to be supported by the system (like "Receive Customer Order") and detailing it through IT-supported (AD) from Fig. 14 (upper part of it). The question that arises here is: does the business use case add any value to the other analysis artifacts? Our answer to it is 'No'. It would be only a textual version of the IT-supported AD and is therefore redundant. It actually changes the context of the business analysis from process flow to the external view of the business system with no credible reason and with the risk of mixing the two contexts. The method of this kind requires two huge leaps: from process flows to IT-supported AD and then straight to system design level of sequence diagramming losing a (architecturally central) use case view of the system on its way.

Full-blown business use case methods replace the Process View of the business system architecture (Fig. 4) with Business Use Case View, which is defined to model the interaction between business Processes and Business Actors [4]. Modeling business processes as such is not an objective of that method. This, in our opinion, is a fundamental difference that is the reason why so many methodologists considers this type of methods practically useless [1, 2, 5, 10] for a purpose of complex business modeling that can be widely accepted by the business community. To illustrate this closer let us once again consider our simple example of the business organization. A business use case model of it would look like the one in Fig. 18.

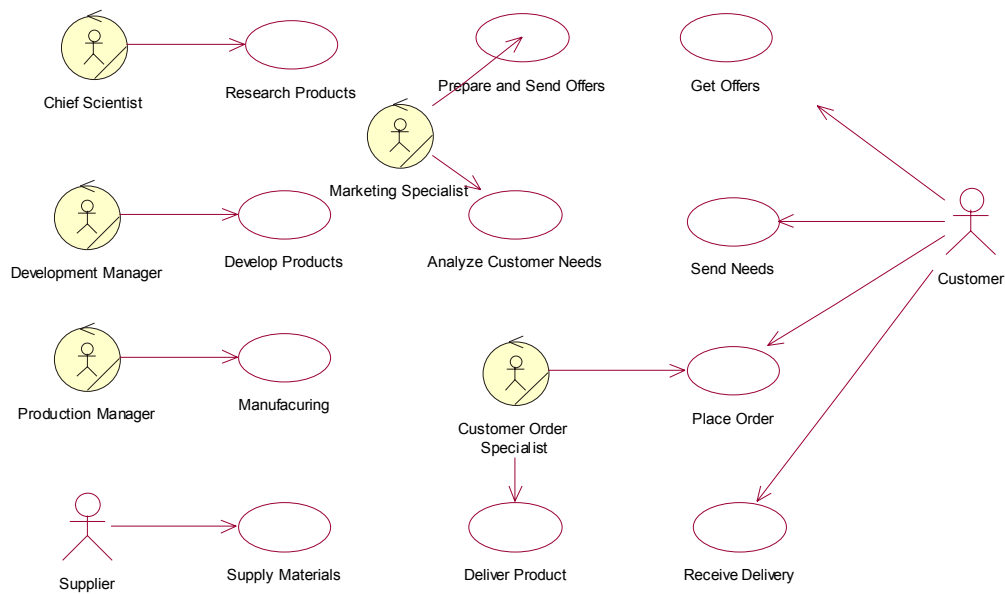


Figure 18. Business Use Case Model of the Example Organisation.

This model is supposed to replace the process map from Fig. 5 and to be used to “drill down into each business use case and analyse how that use case will be performed within the business” [4]. The result of that is to be the set of activity and object diagrams (such as those in Fig. 10 and 11), which belongs to Activity and Object Views as defined in our architecture in Fig. 4. Apart from the fact that all the information from this kind of business use case model is included in the process model, a business use case centred method poses several problems discussed below.

The two models (business use case and business process model) represent different views of the business with no mapping or any other kind of clearly defined association between the two. Assuming customer’s view of the business without focusing on it’s processes and flow of work at different levels can lead to omissions of some critical processes (especially from support and management categories) that are not directly involved in satisfying customer’s request (e.g. ‘Research & Development’ or even ‘Manufacturing’). Without a model of business processes themselves and mapping between them and business use case model we never know whether the later is only fragmentary description of the business or covers all the processes of it as required. Building the process model, in turn, makes the use case model redundant as the former contains all the information of the later and much more. This again makes the applicability of business use case centred architecture of the business - questionable.

Looking at the models from Fig. 18 and Fig. 10 & 11 it’s not difficult to find some other problems to be aware of.

External processes and events from Fig. 5 have become use cases internal to the business. Adding to our model other use cases like ‘Research & Development’ and ‘Manufacturing’ we can see that not only we have to define actors for them (even though it’s still much a higher level of analysis that activity level and some of those actors may simply not make much sense in the context of a use case), but those actors (e.g. Chief

Scientist, Production Manager etc.) are actually Business Workers not Business Actors [16]. This mixture of external and internal context in the business use case analysis is impossible to avoid and can make it quite confusing for even an experienced business analyst.

Looking at the Business Actors, Workers and Use Cases in diagram from Fig. 18 it's not evident, like in Fig. 5, which organisational units they belong to, but even worse than that is the inability of this diagram to show us the flow of work at this level of analysis (sequencing, object flows etc.) – all that it shows is a number of business use cases and business actors and workers interacting with them from the 'outside'. Whether this diagram covers all the business processes of the organisation is not as visible as with the process model of the business. In fact this type of diagram does not contain much of visual modeling at all, contrary to one of the main 'best practices' recommended by all modeling methods – most of the information about the process is written in business use case text. In other words: instead of modelling business processes, we model interactions of the business with its environment. All this makes business use case modelling rather unwieldy as a tool for business process reengineering purpose.

Unlike business processes, business use cases (external view of the business) are not supposed to be decomposed into more granular level, so the next step in business use case driven analysis is detailing them through activity diagrams [4]. This approach uses just two levels of modeling the business instead of as many as practical depending on the complexity of the business being modelled. Activity diagrams are considered an internal view of the business, but it's not clear whether their swimlanes are supposed to correspond to organisational units or business actors/workers interacting with the business use cases defined in the other view. Where to include IT-system into both types of diagrams is not well defined, also. With that, transitioning the business model of this kind into system model is not as clearly defined as with the process centred modelling. Business process centred architecture has been always used to model business processes and is widely accepted and well understood by the business community. Since, as shown in the article, this type of architecture can be easily expressed with UML notation and concepts, the business analysts can model the business the way they so far consider the best (modeling business processes) using UML diagrams. As described above, the deliveries of business modeling of this kind can be then easily transferred to use case centred software architecture that in turn is commonly used by system analysts to define software requirements. In author's opinion this type of methodology applied on the project (process centred business architecture and use case centred software architecture) can prove to be easier to accept for both business and IT teams then trying to convince business community they should model business use cases rather than processes, especially when benefits of this significant change of focus are questionable.

References

1. R. Adhikari, Putting the business in Business Process Modeling, Application Development Trends Magazine White Paper, downloaded from <http://www.adtmag.com/>
2. S.W. Ambler, The Object Primer: Introduction to Techniques for Agile Modeling, Ronin International White Paper, downloaded from <http://www.ronin-intl.com/>
3. B. Baker, Business Modeling with UML: The Light at the End of the Tunnel, Rational Software White Paper, downloaded from <http://www.rational.net/>
4. Business Modeling with the UML and Rational Suite AnalystStudio, Rational Software White Paper, downloaded from <http://www.rational.net/>
5. H.E. Eriksson, M. Penker, Business Modeling with UML, Rational Software White Paper, downloaded from <http://www.rational.net/>
6. H.E. Eriksson, M. Penker, Business Modeling with UML, Business Patterns at Work, J. Wiley & Sons, 2000.
7. R. Grass, Agile versus Traditional, Make Love not War, downloaded from <http://www.agilealliance.com/>
8. J. Heumann, Introduction to Business Modeling Using the Unified Modeling Language, Rational Software White Paper, downloaded from <http://www.rational.net/>
9. C. Marshal, Enterprise Modeling with UML, Designing Successful Software through Business Analysis, Addison-Wesley, 2000.
10. M. McGregor, Modeling the Development Process, Popkin Software White Paper, downloaded from <http://www.popkin.com/>
11. E. McSheffrey, Integrating Business Process Models with UML System Models, Popkin Software White Paper, downloaded from <http://www.popkin.com/>
12. Model Driven Architecture (MDA), Document - ormsc/01-07-01, downloaded from <http://www.omg.org/>
13. I. Jakobson, M. Cristoferson, P. Jonsson, G. Övergård, Object-Oriented Software Engineering – A Use Case Driven Approach, Addison-Wesley, 1992
14. Z. Jackowski, Introducing Extreme System Analysis, The Rational Edge, 05/02. http://www.therationaledge.com/content/may_02/t_extremeAnalysis_zj.jsp
15. P. Kruchten, "The 4+1 View Model of Architecture," IEEE Software, 12 (6), November 1995, IEEE, pp. 42-50.
16. Pan-Wei Ng, Effective Business Modeling with UML: Describing Business Use Cases and Realizations, The Rational Edge, 11/02. http://www.therationaledge.com/content/nov_02/t_businessModelingUML_pn.jsp
17. P. Reed, Transitioning from Requirements to Design, The Rational Edge, 06/02. http://www.therationaledge.com/content/jun_02/m_requirementsToDesign_pr.jsp
18. G.A. Rummler, A.P. Brache, Improving Performance, How to Manage White Space on the Organization Chart, J. Wiley & Sons, 1995 (second edition).
19. Unified Modeling Language (UML), version 1.4, downloaded from <http://www.omg.org/>
20. J. Siegel, Making the Case: OMG's Model Driven Architecture, SD Times, Oct. 15, 2002, <http://www.sdtimes.com/news/064/special1.htm>
21. J. Wardley, Tighten Business Processes Before You Automate, Rational Software White Paper, downloaded from <http://www.rational.net/>